

Verificación y Validación

Práctica 3. Diseño de las pruebas de unidad

Autores:

Daniel Blasco Labarta(900037),

Jorge Bellido Lobera(903080)

Imad Habib Jali(901421)

Particiones de equivalencia del método frecuencias()

El método `frecuencias()` no tiene parámetros explícitos, pero sí uno implícito (**this**, el objeto sobre el que se invoca el método), que tiene dos atributos: `fichero` (objeto de la clase `file` que representa el nombre del fichero) y `frecuencias` (vector de enteros que almacenará el número de apariciones de cada letra en el fichero tras la primera invocación al método `frecuencias()`).

En esta sección se definen las particiones. Para ello, se evalúan las condiciones operativas de sus dos atributos principales: la validez de la referencia al fichero de texto y el estado de inicialización del vector de resultados.

Parámetros	Particiones de equivalencia válidas		Particiones de equivalencia no válidas	
<code>this.fichero</code>	A	Referencia a un objeto <code>File</code> que representa un fichero existente y con permisos de lectura	B	Referencia a un objeto <code>File</code> de un fichero no existente
			C	Ruta a un directorio (no un fichero).
			D	Referencia nula
<code>this.frecuencias</code>	E	Atributo no inicializado (corresponde a la primera invocación del método <code>frecuencias()</code>)		
	F	Atributo ya inicializado con un vector de 27 componentes (corresponde a invocaciones sucesivas)		

Particiones de equivalencia del contenido

Para garantizar una cobertura exhaustiva del procesamiento de texto, se han identificado clases de equivalencia relativas al contenido interno de los ficheros analizados. Este escrutinio distingue de forma sistemática entre los caracteres que deben contabilizarse —incluyendo sus variaciones fonéticas o tipográficas aceptadas— y aquellos elementos que el sistema debe ignorar, tales como símbolos, números o caracteres de puntuación.

Contenido del fichero	Particiones de equivalencia válidas		No válidas	
Letras del alfabeto inglés	I	Caracteres de la 'A' a la 'Z' (mayúsculas y minúsculas) que deben contabilizarse de forma agrupada		
Longitud del contenido	J	Fichero vacío		
	K	Fichero con contenido de texto		
Letra eñe	L	Caracteres 'ñ' y 'Ñ' que deben registrarse en la última posición del vector		
Caracteres no alfabéticos			R	Números, espacios en blanco, signos de puntuación y tabulaciones (no deben ser contados)
Letras con diacríticos, vocales voladas y cedillas	M	Vocales con acento agudo, tanto mayúsculas como minúsculas (á, é, í, ó, ú, Á, É, Í, Ó, Ú) que deben contar como A, E, I, O, U .	S	Consonantes con diacríticos ajenas al alfabeto español/inglés y que no son la cedilla (por ejemplo: š, ĝ, ħ, ċ)
	N	Vocales con acento grave (à, è, ì, ò, ù)	T	Símbolos de diacríticos sueltos o aislados que no están combinados con ninguna letra

				(por ejemplo: ´, ¨, ` , ^)
	Ñ	Vocales con acento circunflejo (â, ê, î, ô, û)		
	O	Vocales con diéresis (ä, ë, ï, ö, ü, Ä, Ë, Ì, Ö, Ü)		
	P	Letra 'c' con cedilla (ç, Ç), que debe contabilizarse como la letra 'C' sin modificaciones .		
	Q	Vocales voladas (ª, º), que deben contabilizarse como 'A' y 'O' respectivamente.		

Análisis de valores límite

Complementando las particiones de equivalencia, se identifican los puntos críticos en las fronteras de las clases de equivalencia, tanto para el fichero como contenedor como para su contenido alfabético.

Elemento	Valor Límite Inferior (VLI)	Valor Límite Superior (VLS)	Justificación
Tamaño del fichero	0 caracteres (Fichero vacío)	Fichero muy grande (Ej: El Quijote)	Verifica el comportamiento del sistema ante la ausencia total de datos y su robustez procesando ficheros muy extensos.
Tamaño mínimo con datos	1 carácter	N/A	Verifica que el bucle de lectura se ejecuta correctamente para el caso mínimo válido (VLI + 1).
Rango de letras del alfabeto (A-Z, a-z)	Letra A / a (Frontera inválida: @ y `)	Letra Z / z (Frontera inválida: [y {)	Verifica la transición exacta en la tabla ASCII comprobando tanto el primer/último elemento válido del rango, como el inmediatamente anterior/posterior o inválido.

Particiones de equivalencia del Main

Para validar la tolerancia a fallos de esta interfaz, se han establecido particiones de equivalencia que modelan los posibles valores de entrada que el usuario puede proporcionar por teclado.

Parámetros	Particiones de equivalencia válidas		Particiones de equivalencia no válidas	
Ruta del fichero	A	Ruta a un fichero de texto existente y con permisos de lectura.	B	Ruta a un fichero que no existe.
			C	Ruta a un directorio (no un fichero).
			D	Entrada vacía (pulsar Intro directamente).

Casos de prueba

A partir de la combinación y simplificación de las clases de equivalencia y los valores límite definidos en los apartados anteriores, se ha derivado el siguiente conjunto formal de casos de prueba unitarios. Estos casos trazan cada escenario con su respectiva entrada simulada y definen el vector de frecuencias exacto que el método debe devolver.

Caso	this.fichero (continente y contenido)	this.frecuencias	Resultado esperado	Clases cubiertas	Comentarios
1	quijote.txt	null	Vector de 27 enteros correspondiente a la salida esperada en src/test/res	A,E,I,K,L,R, Tamaño del fichero (VLS).	Sirve como prueba de estrés y límite superior práctico
2	Referencia a File que apunta a un fichero que no existe	null	FileNotFoundException	B,E	
3	Ruta a un directorio (no un fichero).	null	FileNotFoundException	C,E	
4	Referencia a File que es null	null	NullPointerException	D,E	
5	vacio.txt (Fichero sin ningún contenido)	null	Vector de 27 componentes, todas con valor 0	A,E,J, Tamaño del fichero (VLI).	
6	diacriticos.txt (Contiene caracteres con diacríticos)	null	Vector de 27 enteros con el valor 5 en la posición de cada vocal, valor 1 en la posición de la 'C' (por la ç), valor 1 en la posición de la 'Ñ' (por la ñ) y el resto de posiciones a 0.	A, E, I, K, L, M, N, Ñ, O, P, Q	Aísla vocales modificadas, cedilla y eñe.
7	ignorados.txt (Contiene exactamente: "123 !? ê ´ ¨")	null	Vector de 27 componentes, todas con valor 0	A, D, K, R, S, T	Verifica que se ignoran caracteres no válidos.

8	El mismo objeto ya instanciado y ejecutado en el Caso 7	Vector ya inicializado con las frecuencias del Caso 7	Devuelve exactamente el mismo vector que en el Caso 7	A, F, I, K, L, M, N, Ñ, O, P, Q	Segunda invocación. Verifica que no se recalculan datos.
9	un_caracter.txt	null	Vector con 1 en la posición 0 ('A'). Resto 0.	Tamaño mínimo con datos (VLI+1).	
10	fronteras_alfabeto.txt (Contiene exactamente: @A`aZ[z{)	null	Vector con: 2 en la posición 0 ('A'), 2 en la posición 25 ('Z'). Resto 0.	Rango de letras del alfabeto (VLI y VLS, incluyendo sus fronteras externas).	

Para garantizar la validez y reproducibilidad de los casos de prueba, es un requisito estricto que todos los ficheros de texto generados para estas pruebas sean guardados explícitamente utilizando la codificación UTF-8.

Casos de prueba Main

Caso	Entrada	Resultado Esperado	Clases cubiertas
11	"src/main/res/quijote.txt\n"	El programa solicita el fichero y luego imprime el listado completo desde la A: hasta la Z	A
12	"ruta_inventada_falsa.txt\n"	El programa imprime un mensaje de error controlado indicando El fichero 'f' no existe..	B
13	"src/main/res/\n"	El programa imprime un mensaje de error controlado indicando El fichero 'f' no existe..	C
14	"" (Cadena vacía)	El programa imprime un mensaje de error controlado indicando El fichero 'f' no existe..	D